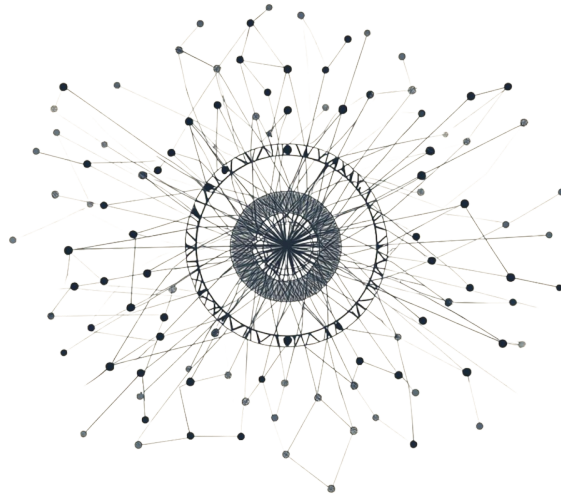




Better Galaxy Classification Using ML

Final Report

Golightly Chamberlain, Ben Derieg, Aditya Mukerjee

April 26

Introduction and Motivation.....	3
Background and Course Connection.....	4
Data Sources and Data Construction.....	5
Data Cleaning and Feature Engineering.....	6
Data Storage and Pipeline.....	7
Modeling Methodology.....	8
Model Evaluation and Results.....	10
Team Contributions.....	14
Conclusion.....	15

Introduction and Motivation

Galaxy morphology classification is important because morphology gives astronomers a compact way to describe the structure and evolutionary state of galaxies. However, many commonly used morphology labels come from legacy catalogs such as [HyperLEDA](#) and de Vaucouleurs' [Third Reference Catalog of Bright Galaxies](#) (RC3). These labels are useful, but they may be incomplete, inconsistent, or outdated, and manually validating large numbers of galaxies is not practical.

The goal of this project is to test whether modern tabular measurements from the [Sloan Digital Sky Survey](#) (SDSS) contain enough signal to classify galaxies as spiral or elliptical. To do this, we built a pipeline that downloads SDSS measurements, joins them with legacy morphology data by sky coordinate, cleans and engineers features, and evaluates multiple machine learning models. The project focuses on whether features such as galaxy color, brightness, surface brightness, and concentration can be used to predict morphology reliably. If these models perform well, they can help identify possible inconsistencies in older labels and provide a more scalable way to study galaxy classification.

Background and Course Connection

This project fits directly into the focus of CS 4964: Manage Data for & with Machine Learning. The course emphasizes the full data lifecycle needed for machine learning, including data ingestion, relational data management, data cleaning, scalable processing, data-centric machine learning, storage layout, visualization, and model evaluation. Our project follows that same structure by building a pipeline that starts with raw astronomy data, joins multiple sources, cleans and engineers features, stores the cleaned dataset, and then uses the resulting data for supervised and unsupervised machine learning.

Several parts of the project connect to early course topics on relational databases, SQL, indexing, and Pandas. The project uses tabular data from SDSS, HyperLEDA, and de Vaucouleurs catalogs, then joins these sources into a single modeling dataset. Although the join is done in Python rather than SQL, it reflects the same relational data-management idea: different tables contain different pieces of information, and the pipeline must align them into one usable dataset. In our case, the key matching problem is not a normal ID join, but a sky-coordinate match between SDSS objects and legacy morphology catalogs.

The project also connects strongly to the course's data-centric machine learning material. Instead of focusing only on model tuning, much of the project work involved improving the dataset itself. We expanded the data pipeline with HyperLEDA, added new features, handled missing values, removed outliers, and tracked which values were imputed. This reflects the course idea that model performance depends heavily on the quality, structure, and reliability of the training data.

The modeling portion connects to the course's machine learning review, classification, clustering, and dimensionality reduction topics. For supervised learning, we compare Logistic Regression, Gradient Boosting, and Random Forest models on the same cleaned feature set. For unsupervised learning, we use K-Means clustering and PCA visualization to test whether the feature space naturally separates into morphology-like groups without using the labels during training.

Finally, the project connects to the course topics on storage layout and visualization. The cleaned dataset is saved as a Parquet file, which is appropriate for column-oriented analytical workflows, and can also be exported to PostgreSQL for relational querying. The final analysis produces confusion matrices, a model metric comparison chart, a K-Means elbow plot, and a PCA cluster visualization. These outputs make the pipeline results easier to inspect and communicate, which matches the course emphasis on using visualization to understand large datasets and machine learning behavior.

Data Sources and Data Construction

The dataset for this project is built from three main sources: SDSS photometric measurements, HyperLEDA catalog data, and de Vaucouleurs morphology data. SDSS provides the modern measurements used as model features, while HyperLEDA and de Vaucouleurs provide legacy morphology information used to construct the target label. The project does not use galaxy images directly. Instead, it tests whether tabular measurements such as color, brightness, surface brightness, and concentration are enough to predict whether a galaxy is spiral or elliptical.

The first stage of the pipeline collects SDSS data. The `get_sdss_data.py` script queries the SDSS PhotoObj table for objects classified as galaxies using `type = 3`, and limits the sample to galaxies with $r < 19$ (i.e., r-band brightness above a certain threshold). The query pulls each galaxy's sky coordinates (`ra`, `dec`), SDSS filter magnitudes/brightnesses (`u`, `g`, `r`, `i`, `z`), and Petrosian measurements (`petroMag_r`, `petroR50_r`, `petroR90_r`; alternative brightness and size quantifiers). To avoid putting too much load on the SDSS servers, the script queries the data in right-ascension bins, combines the results, and saves them locally as `sdss_raw.parquet`.

The second stage joins the SDSS measurements to legacy morphology catalogs. The pipeline reads de Vaucouleurs data from `devac_raw.tsv` (downloaded from the original paper) and HyperLEDA data from `leda_raw.tsv` (downloaded from the HyperLEDA website). The de Vaucouleurs file contributes a morphology value called `T_devac`, while HyperLEDA contributes `T_leda` along with additional physical features such as `logdc` (log of central surface brightness), `bri25` (surface brightness along the outer edge), and `mabs` (absolute/intrinsic magnitude). Since the raw HyperLEDA file is too large to store directly in the repository, users download it separately and place it in the `data/` directory before running the pipeline.

The join is based on sky-coordinate matching rather than a shared ID. This is necessary because the catalogs do not all use the same object identifiers. The pipeline converts the SDSS, HyperLEDA, and de Vaucouleurs coordinates into coordinate objects, then matches each SDSS galaxy to the nearest object in the legacy catalogs. Matches farther than five arcseconds are rejected to reduce the chance of linking the wrong galaxy, and the closest match is selected in the rare case that there are two matches within 5 arcseconds. After matching, the SDSS table is joined with both morphology sources, and rows without a match in either legacy catalog are removed.

The result of this stage is a combined table saved as `data_full.parquet`. This joined dataset contains modern SDSS measurements, legacy morphology labels, and HyperLEDA-derived physical features. It becomes the input to the cleaning and feature-engineering stage, where the final model-ready dataset is created.

Data Cleaning and Feature Engineering

After the raw SDSS, HyperLEDA, and de Vaucouleurs data are joined, the next stage is to turn the combined table into a clean modeling dataset. The cleaning script starts by combining the two available morphology values, `T_leda` and `T_devac`, into one numeric morphology column called `T`. Rows without a valid morphology value are removed because they cannot be used for supervised training. The final binary target label, morphology, is then derived from `T`: galaxies with $T < 0$ are labeled as elliptical, while the remaining galaxies are labeled as spiral. This is according to the definition of `T` in these catalogs.

The script also converts key numeric columns into proper numeric types. This step is necessary because raw catalog files can contain missing values, formatting issues, or values that are read as strings instead of numbers. Columns such as sky coordinates, SDSS magnitudes, Petrosian measurements, and HyperLEDA features are cast into numeric form before feature engineering is performed.

Several new features are then created from the raw measurements. The pipeline calculates `sb50_r`, which represents the mean surface brightness in the central region of the galaxy. It also calculates `sb_conc_r`, a concentration feature based on the ratio between `petroR90_r` and `petroR50_r`. In addition, the script creates color features by subtracting SDSS filter magnitudes from each other. These include `ug_color`, `ur_color`, `ui_color`, `uz_color`, `gr_color`, `gi_color`, `gz_color`, `ri_color`, `rz_color`, and `iz_color`. These engineered features are important because galaxy color, brightness, and concentration are expected to relate to morphology.

The cleaning step also removes outliers from the main numeric modeling features. For each selected feature, the script calculates the interquartile range and removes values outside the normal IQR bounds. This helps reduce the effect of extreme or likely unreliable values in the final model input. After outlier filtering, columns that are not useful for modeling are dropped, including raw catalog IDs, intermediate coordinate columns, original SDSS magnitude columns, and other fields that were only needed during joining or feature construction.

Missing-value handling is focused on the three HyperLEDA-derived features `logdc`, `bri25`, and `mabs`. Instead of imputing every feature, the pipeline only imputes these selected columns. This is because our data sources are complete in every other column. Before imputation, the script creates indicator columns named `imputed_logdc`, `imputed_bri25`, and `imputed_mabs`. These columns record whether the original value was missing. Then, a KNN imputer fills the missing values for `logdc`, `bri25`, and `mabs`. This lets the model use the completed feature values while still preserving information about which values were originally imputed. Less than 10% of values in these columns were imputed.

The final cleaned dataset is saved as `data_full_cleaned.parquet`. In the final run, this dataset contained 148,968 rows and 18 columns, with 84,415 spiral galaxies and 64,553 elliptical galaxies. This dataset is the main input for both the supervised model evaluation pipeline and the unsupervised clustering analysis.

Data Storage and Pipeline

The project uses a source-file pipeline so the full workflow can be rerun from the command line instead of relying on notebook cells. The main entry point is `clean_and_load_data.sh`, which runs the project scripts in order: `get_sdss_data.py`, `join_morph_sdss.py`, `clean_data.py`, `evaluate_models.py`, and `cluster_analysis.py`. This gives the project a repeatable workflow from raw data collection through final model and clustering outputs.

The pipeline uses Parquet as the main file-based storage format. The raw SDSS query output is saved as `sdss_raw.parquet`, the joined SDSS and morphology data is saved as `data_full.parquet`, and the final cleaned modeling dataset is saved as `data_full_cleaned.parquet`. Parquet is useful here because the project works with tabular, column-oriented data, and the same cleaned file can be loaded directly into Pandas for modeling, visualization, and clustering.

In addition to Parquet, the project supports PostgreSQL export. After cleaning, `clean_data.py` attempts to write the final cleaned dataset into a PostgreSQL table named `galaxy_data_cleaned`. This allows the cleaned galaxy data to be inspected and queried with SQL instead of only being available as a local Parquet file. PostgreSQL is not required for the rest of the pipeline to work, but it gives the project a more flexible storage option and connects the cleaned dataset to a relational database workflow.

The PostgreSQL export uses a default local connection string, but it can also use a custom `DATABASE_URL` environment variable. If PostgreSQL is running locally, the export writes the cleaned rows into the database table. If the database connection fails, the pipeline still saves the cleaned Parquet file and continues to preserve the main data output. This makes PostgreSQL useful but not fragile: it adds database support without making the whole pipeline fail if a local database is not configured.

After the cleaned dataset is saved, the pipeline continues into analysis. `evaluate_models.py` loads `data_full_cleaned.parquet` and runs the supervised model comparison, saving model metrics and plots into the `results/` directory. `cluster_analysis.py` also loads the cleaned dataset and produces K-Means and PCA outputs in the same results directory. This keeps the project outputs organized: data files are stored in `data/`, while model results, clustering results, CSV summaries, and plots are stored in `results/`.

Modeling Methodology

The modeling stage uses the final cleaned dataset, `data_full_cleaned.parquet`, as the input for both supervised classification and unsupervised clustering. The supervised task is to predict the binary morphology label, where galaxies are labeled as either spiral or elliptical. The input features are the cleaned physical and photometric measurements produced during feature engineering, including surface brightness, concentration, SDSS color features, HyperLEDA-derived brightness features, and the imputation indicator columns.

For supervised learning, the project compares three classifiers: Logistic Regression, Gradient Boosting, and Random Forest. These models were chosen because they represent different approaches to classification. Logistic Regression acts as a simple linear baseline. It helps show how well a relatively simple decision boundary can separate spiral and elliptical galaxies. Gradient Boosting provides a stronger ensemble method that builds sequential decision trees to correct earlier errors. Random Forest provides another ensemble approach by training many decision trees and combining their predictions. This gives a useful comparison between a linear model and two tree-based models.

The supervised models are trained and evaluated using the same pipeline in `evaluate_models.py`. The dataset is split into training and testing sets using an 80/20 train-test split with stratification, so the spiral/elliptical class balance is preserved in both sets. Logistic Regression is placed inside a pipeline with feature scaling because it is sensitive to differences in feature magnitude. The tree-based models, Gradient Boosting and Random Forest, are trained directly on the feature values because they do not require the same scaling step.

The model comparison uses the same feature columns and test set for every classifier. This is important because it makes the results directly comparable. Each model is trained on the training data, used to predict labels for the test data, and then passed through a shared evaluation function. That function calculates RMSE, accuracy, balanced accuracy, precision, recall, and F1 score. It also prints a classification report, saves a confusion matrix plot, and adds the model's scores to a shared results table.

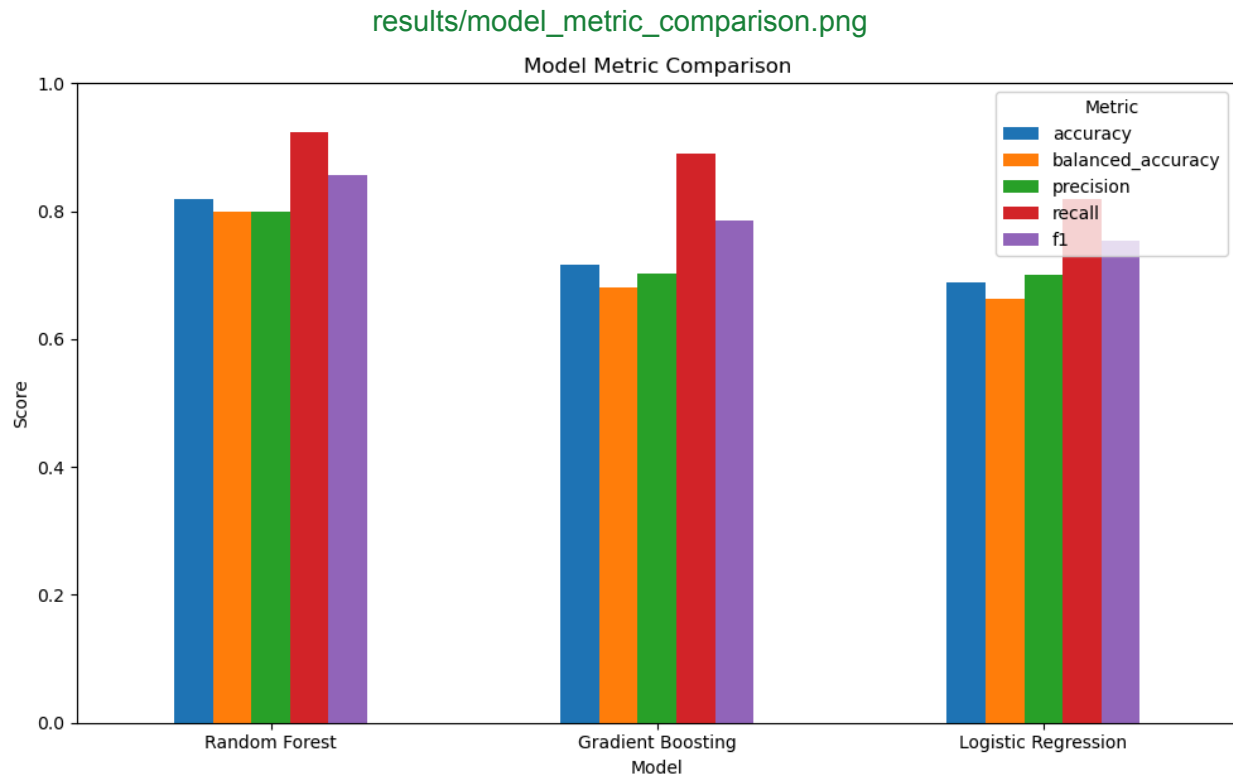
The project also includes an unsupervised learning component using K-Means clustering. Unlike the supervised models, K-Means does not use the morphology labels during training. Instead, it groups galaxies based only on their scaled feature values. The main K-Means run uses $k=2$ because the cleaned target labels are binary: spiral and elliptical. The script also runs an elbow analysis from $k=2$ through $k=6$ to check whether the feature space suggests more than two natural clusters. After clustering, the predicted clusters are compared back to the known morphology labels to see whether the unsupervised groups align with the legacy classifications.

Finally, PCA is used to visualize the K-Means clusters in two dimensions. Since the modeling dataset has many numeric features, PCA gives a way to project the scaled feature space into two principal components for plotting. This visualization is not used as a classifier, but it helps

interpret whether the K-Means clusters form visible structure and whether those clusters appear to correspond to known morphology groups.

Model Evaluation and Results

The supervised models were evaluated on the same held-out test set using RMSE, accuracy, balanced accuracy, precision, recall, and F1 score. Accuracy was useful as a general performance measure, but we also included balanced accuracy and F1 because the dataset is moderately imbalanced, with more spiral galaxies than elliptical galaxies. This made it important to check whether a model was actually learning both classes instead of mainly predicting the majority class.

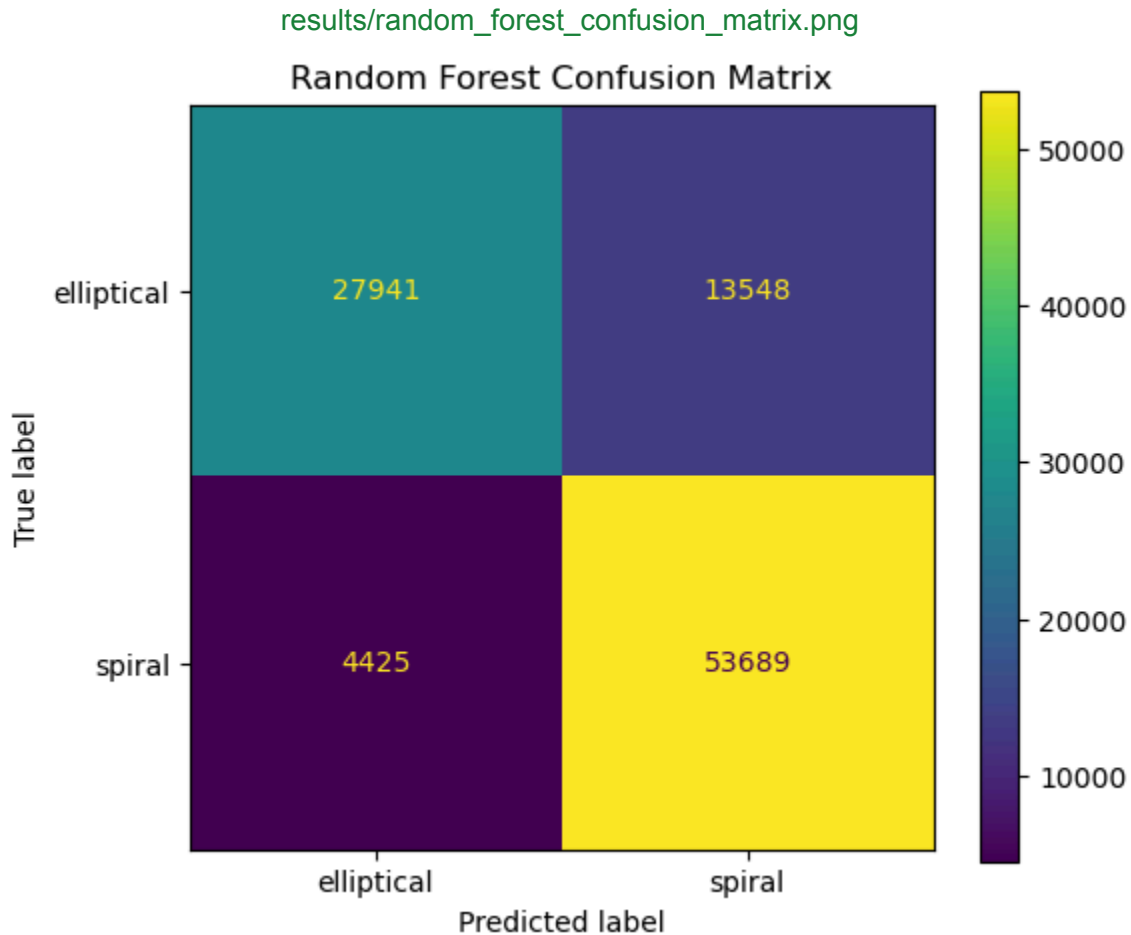


The overall comparison showed that **Random Forest was the strongest supervised model**. Random Forest achieved the best score across the main evaluation metrics, with an accuracy of **0.8196**, balanced accuracy of **0.7987**, precision of **0.7985**, recall of **0.9239**, F1 score of **0.8566**, and RMSE of **0.4248**.

Gradient Boosting was the second-best model, with an accuracy of **0.7160**, balanced accuracy of **0.6811**, precision of **0.7025**, recall of **0.8902**, F1 score of **0.7853**, and RMSE of **0.5329**. Logistic Regression performed the weakest of the three, with an accuracy of **0.6893**, balanced accuracy of **0.6634**, precision of **0.6999**, recall of **0.8186**, F1 score of **0.7546**, and RMSE of **0.5574**.

The confusion matrices show the same pattern in more detail. Gradient Boosting correctly predicted many spiral galaxies, but it struggled with elliptical galaxies and misclassified **21,906 elliptical galaxies as spiral**. Logistic Regression improved this issue, reducing

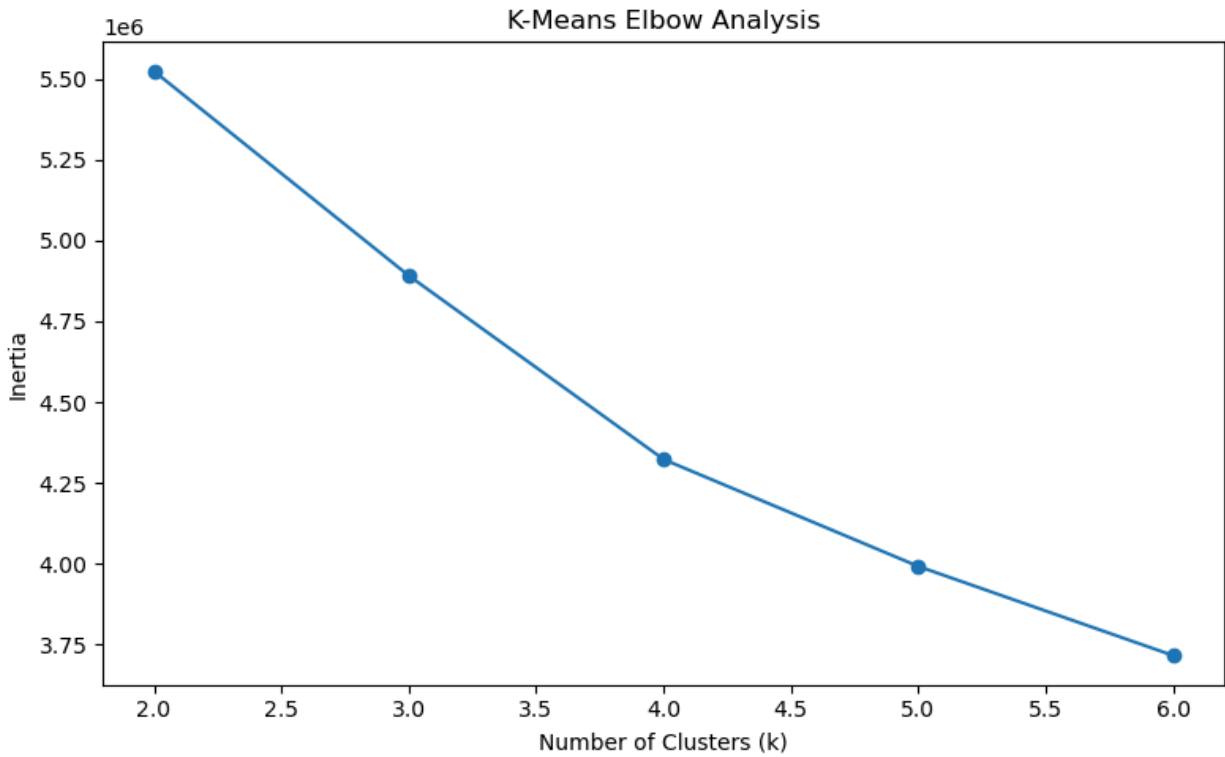
elliptical-to-spiral mistakes to **6728**. Random Forest performed best, reducing that error further to **13,548** while also correctly classifying **53,689 spiral galaxies** and **27,941 elliptical galaxies**.



The Random Forest result suggests that the engineered tabular features contain real morphology signal. The model was able to use color, brightness, surface brightness, concentration, and HyperLEDA-derived features to distinguish spiral and elliptical galaxies much more effectively than the linear Logistic Regression baseline. This also suggests that the relationship between the features and morphology is likely not purely linear, since the tree-based models performed substantially better.

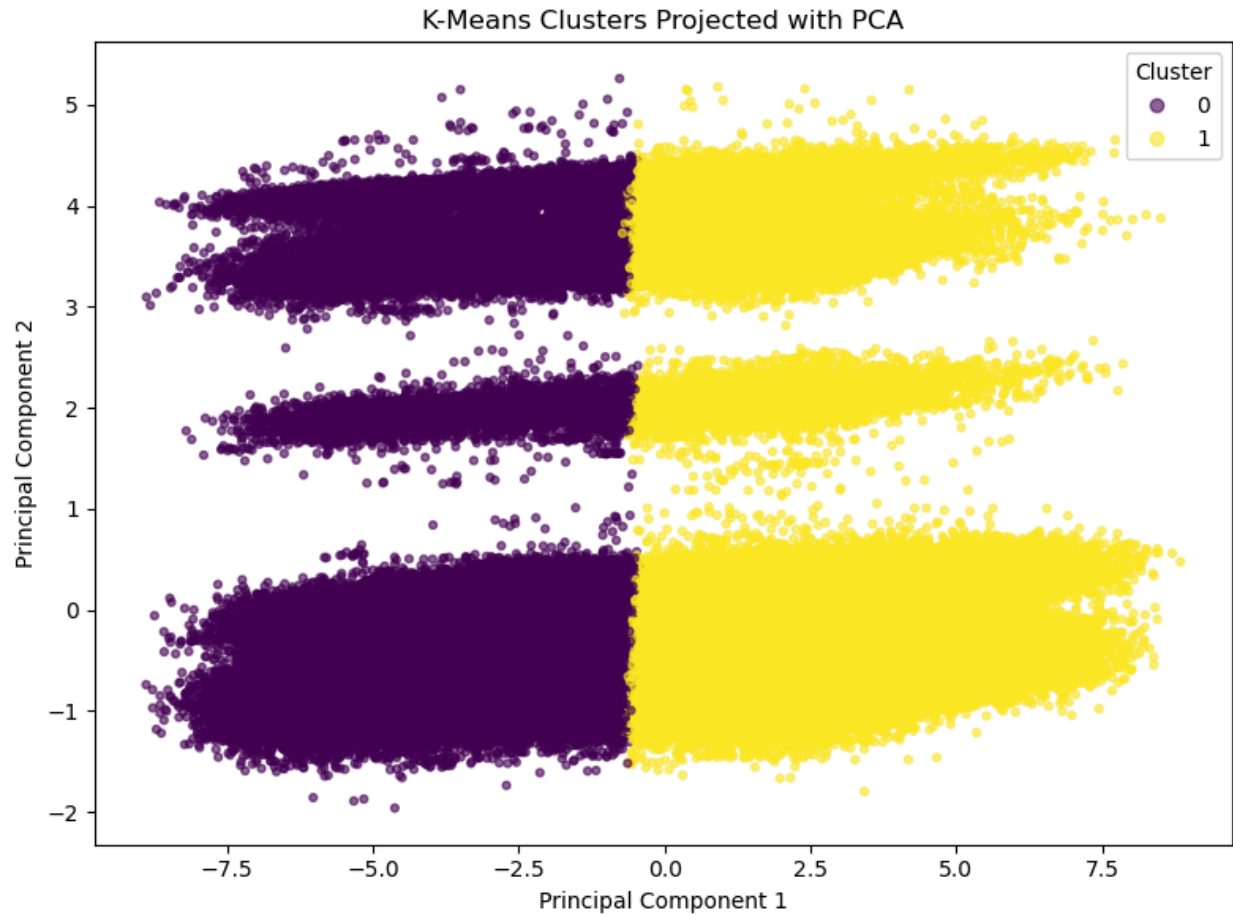
The unsupervised K-Means results were different. K-Means was run with $k=2$ as the main clustering comparison because the cleaned morphology labels are binary. The elbow analysis tested values from $k=2$ through $k=6$ to check whether the data suggested a stronger natural cluster count.

results/kmeans_elbow_plot.png



After running K-Means with two clusters, the clusters did not align well with the known spiral and elliptical labels. Cluster 0 contained about **41.55% elliptical** and **58.45% spiral** galaxies, while Cluster 1 contained about **41.72% elliptical** and **58.28% spiral** galaxies. Since both clusters had similar class proportions, K-Means did not naturally separate the galaxies into one spiral-heavy group and one elliptical-heavy group. The Adjusted Rand Index was **-0.0001**, which is essentially no agreement between the K-Means clusters and the known morphology labels.

results/kmeans_pca_clusters.png



The PCA cluster visualization still showed that K-Means found some structure in the scaled feature space, but that structure did not correspond strongly to the spiral/elliptical labels. This is an important result because it shows a difference between supervised and unsupervised learning on this dataset. Supervised models, especially Random Forest, could learn predictive boundaries from the labeled examples, but unsupervised K-Means did not independently recover the same morphology categories from the feature space alone.

Overall, the final results support the main project goal. The supervised models show that modern tabular galaxy measurements can predict legacy morphology labels with meaningful accuracy, especially when using Random Forest. However, the unsupervised clustering results suggest that the morphology distinction is not represented as a simple two-cluster structure in the engineered feature space. This means the data contains useful predictive signal, but that signal requires supervised learning rather than simple unsupervised clustering to recover effectively.

Team Contributions

Aditya worked on the initial modeling direction for the project. His work helped establish Random Forest as the first main supervised baseline and identified KNN, optimized KNN, weighted KNN, and K-Means as additional model directions to explore.

Ben focused on the data pipeline and dataset construction work. He expanded the project beyond the smaller midterm dataset by incorporating HyperLEDA into the pipeline and joining it with SDSS and de Vaucouleurs morphology data. This substantially improved the available modeling dataset and added new HyperLEDA-derived features, including logdc, bri25, and mabs. He also updated the cleaning strategy so the project now saves one final cleaned dataset instead of maintaining separate imputed and non-imputed versions. In addition, he revised missing-value handling by trimming rows with missing core brightness/color data while still imputing selected HyperLEDA features and adding indicator columns to show which values were imputed.

Golightly focused on making the project more reproducible, runnable, and evaluation-ready. He added PostgreSQL export support to the cleaned data pipeline, including a default local database connection and fallback behavior so the cleaned Parquet output is still preserved if database export fails. He also implemented the final supervised model evaluation pipeline in `evaluate_models.py`, including Logistic Regression, Gradient Boosting, Random Forest integration, shared train/test splitting, shared evaluation metrics, confusion matrix plots, model comparison output, and best-model selection. Finally, he implemented the unsupervised clustering pipeline in `cluster_analysis.py`, including K-Means elbow analysis, K-Means clustering with $k=2$, PCA visualization, cluster-to-morphology comparison, and Adjusted Rand Index reporting.

Conclusion

This project built a full pipeline for galaxy morphology classification using SDSS measurements joined with legacy HyperLEDA and de Vaucouleurs morphology data. The final workflow downloads and joins raw data, cleans and engineers features, saves the cleaned dataset, exports to PostgreSQL when available, and produces both supervised and unsupervised model results.

The supervised results show that the engineered tabular features contain meaningful morphology signal. Random Forest performed best, outperforming Logistic Regression and Gradient Boosting across the main evaluation metrics. The unsupervised K-Means results were less aligned with the known labels, suggesting that spiral and elliptical galaxies are not separated into two simple clusters in the current feature space.

Overall, the project shows that modern tabular galaxy measurements can be used to predict legacy morphology labels reasonably well, especially with tree-based supervised models. It also creates a reproducible source-file pipeline that can be extended with additional models, stronger validation, and future misclassification analysis.